

ErrorOps - AI 기반 Kubernetes 장애 분석 및 코드 수정 자동화 플랫폼

에 분석 서비스로 Alert를 전달합니다. ai-error-analyzer는 Kubernetes Logs와 Events를 수집한 뒤 Gemini API를 활용해 장애 원인 분석과 수정 작업 프롬프트를 생성하고, Slack 승인 기반 Local Codex Runner가 해당 프롬프트를 실행해 코드 수정 후보 작성, 브랜치 생성, 커밋, Pull Request 생성까지 수행하는 장애 대응 자동화 플랫폼

기술 스택

IaC

- Terraform
- AWS Provider
- S3 Backend
- S3 Native Lockfile

Cloud Infrastructure

- AWS VPC
- AWS EKS
- AWS RDS MySQL
- AWS ElastiCache Redis
- AWS ECR
- AWS IAM
- AWS Secrets Manager
- AWS Systems Manager Session Manager
- AWS Load Balancer Controller

Container & Orchestration

- Docker
- Kubernetes
- Amazon EKS
- Kubernetes Service
- Kubernetes Ingress
- HPA
- Kubernetes RBAC

GitOps & CI/CD

- GitHub Actions
- ArgoCD
- Kustomize

Monitoring & Observability

- Prometheus
- Grafana
- Alertmanager
- kube-prometheus-stack

Backend - AI Analyzer

- Java 17
- Spring Boot
- Kubernetes Java Client
- Gemini API
- Slack Webhook
- Slack Bot API
- PostgreSQL

Backend - Messenger

- Java 17
- Spring Boot
- Web Socket
- Rabbit MQ

Automation - Local Codex Runner

- Node.js
- Slack Web API
- Codex CLI
- GitHub CLI

Security & Secret Management

- IAM Role
- IRSA
- External Secrets Operator
- AWS Secrets Manager
- Kubernetes ServiceAccount
- Kubernetes RBAC
- GitHub OIDC

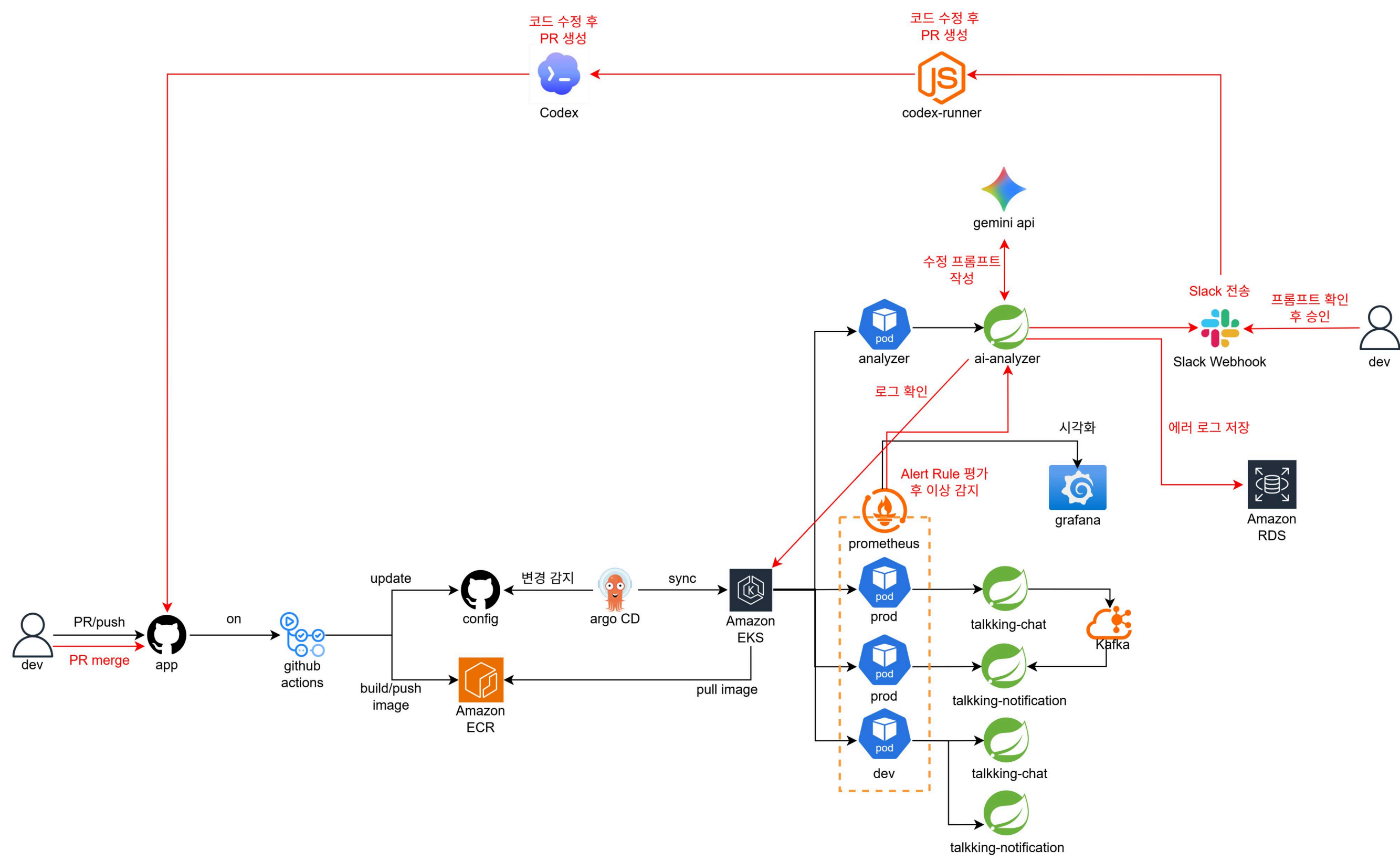
Collaboration

- Git
- GitHub
- Notion
- Slack

프로젝트 개요

- **프로젝트명**
ErrorOps - AI 기반 Kubernetes 장애 분석 및 코드 수정 자동화 플랫폼
- **프로젝트 소개**
AWS EKS 기반 애플리케이션 운영 환경에서 Kubernetes 장애를 자동으로 감지하고, AI를 활용해 장애 원인 분석부터 수정 제안, Pull Request 생성까지 연결하는 장애 대응 자동화 플랫폼입니다. Terraform을 활용해 AWS 인프라를 구성하고, GitHub Actions와 ArgoCD 기반 GitOps 배포 파이프라인을 구축했습니다. Prometheus, Alertmanager, Grafana를 활용해 Pod 상태, 재시작 횟수, CPU/Memory, 응답시간, 5xx 에러율 등을 모니터링하고, 감지된 Alert를 ai-error-analyzer로 전달해 Kubernetes Logs와 Events를 수집하도록 설계했습니다. 이후 Gemini API가 장애 분석 결과와 Codex 작업 프롬프트를 생성하고, Slack 승인 기반 Local Codex Runner가 수정 후보 작성, 브랜치 생성, 커밋, Pull Request 생성까지 수행하는 반자동 장애 대응 흐름을 구현했습니다.
- **개발 인원**
5명
- **담당 역할**
 - **Team Leader**
 - i. 역할 분배
 - ii. 프로젝트 일정 관리
 - iii. 시스템 흐름 및 기능 설계
 - iv. ArgoCD 기반 GitOps 배포 구조 설계
 - **Cloud Infrastructure**
 - i. Terraform 기반 AWS 인프라 구성
 - ii. VPC, EKS, RDS, ElastiCache Redis, ECR 리소스 구성
 - iii. S3 Backend 기반 Terraform 원격 상태 관리 구성
 - iv. IAM Role, IRSA, GitHub OIDC 권한 구성
 - **Monitoring & ErrorOps**
 - i. PrometheusRule 기반 Pod 장애 감지 룰 작성
 - ii. AlertmanagerConfig를 통한 ai-error-analyzer Webhook 라우팅 구성
 - iii. Kubernetes Logs 및 Events 수집 기반 장애 분석 흐름 설계
 - iv. Gemini API 기반 장애 분석 및 Codex 작업 프롬프트 생성 기능 구현
 - v. Slack Webhook 및 Slack Bot 기반 심각도별 알림, 승인 흐름 구성
 - vi. Local Codex Runner를 통한 코드 수정 자동화 및 Pull Request 생성 흐름 구현

기능흐름



- 1. 애플리케이션 코드 변경:
개발자가 App Repository에 코드를 Push하거나 PR을 생성
- 2. CI 파이프라인 실행:
GitHub Actions가 자동 실행되어 애플리케이션 빌드 및 테스트 수행
Docker 이미지를 생성한 후 Amazon ECR에 Push
- 3. GitOps 배포 트리거:
GitHub Actions가 Config Repository의 이미지 태그를 최신 버전으로 업데이트
ArgoCD가 Config Repository의 변경 사항을 감지
- 4. 애플리케이션 배포:
ArgoCD가 변경된 Kubernetes Manifest를 Amazon EKS에 동기화(Sync)
EKS는 Amazon ECR에서 최신 이미지를 Pull하여 애플리케이션을 배포
- 5. 모니터링 및 메트릭 수집:
Prometheus가 EKS 클러스터와 애플리케이션의 Pod 상태, 재시작 횟수, CPU/Memory, Kubernetes 상태 메트릭을 주기적으로 수집
Grafana는 Prometheus 메트릭을 기반으로 클러스터 및 애플리케이션 상태를 시각화
- 6. 장애 감지:
PrometheusRule이 수집된 메트릭을 기반으로 장애 조건을 평가
CrashLoopBackOff, Pod 재시작 증가, Pod NotReady 등의 조건이 충족되면 Alert가 pending 상태가 되고, 설정된 시간 동안 지속되면 firing 상태로 전환
- 7. Alertmanager 라우팅:
Prometheus에서 발생한 Alert는 Alertmanager로 전달
AlertmanagerConfig는 Alert를 ai-error-analyzer Webhook Receiver로 라우팅
- 8. AI 장애 분석 요청:
Alertmanager가 ai-error-analyzer의 alerts 엔드포인트로 Alert 정보를 전송
ai-error-analyzer는 Alert에 포함된 namespace, pod, container, severity 정보를 기반으로 장애 대상 Pod를 식별
- 9. Kubernetes 로그 및 이벤트 수집:
ai-error-analyzer가 Kubernetes API를 호출하여 장애 Pod의 최근 로그와 Events를 조회
ImagePullBackOff, OOMKilled, Readiness Probe 실패, FailedScheduling 등 장애 원인 판단에 필요한 이벤트 정보를 함께 수집
- 10. AI 기반 분석 및 수정 프롬프트 생성:
ai-error-analyzer가 Alert 정보, Pod 상태, Kubernetes Logs, Events를 Gemini API에 전달
Gemini는 장애 원인 분석, 영향 범위, 확인 명령어, 조치 방향을 생성
동시에 Local Codex Runner가 실행할 수 있는 코드 수정 작업 프롬프트를 생성

11. Slack 알림 및 승인 요청:

ai-error-analyzer는 Gemini API에 장애 정보와 Kubernetes 로그를 전달
Gemini API는 장애 원인, 영향 범위, 확인 명령어, 권장 조치가 포함된 런북을 생성

12. Local Codex Runner 실행:

Node.js 기반 Local Codex Runner가 Slack 채널을 주기적으로 polling
[CODEX_TASK] 메시지와 승인 reaction 개수를 확인한 뒤, 승인 조건이 충족되면 첨부된 프롬프트 파일을 다운로드
Safety Guard로 위험 명령어를 검사한 후 로컬 Codex CLI를 실행

13. 코드 수정 자동화:

Codex CLI가 app/config/infra Repository를 확인하고, 장애 원인에 따라 필요한 파일만 최소 범위로 수정
새 브랜치를 생성하고 변경 사항을 커밋한 뒤 원격 Repository에 Push
dev 브랜치에 직접 반영하지 않고 Pull Request를 생성하여 사람의 검수 단계 유지

14. 검수 및 반영:

운영자 또는 개발자가 Pull Request의 변경 사항을 검토
문제가 없으면 PR을 Merge하여 Config 또는 App Repository에 반영
이후 GitHub Actions와 ArgoCD를 통해 수정 사항이 다시 EKS에 배포

15. 장애 해소 확인:

Prometheus와 Grafana를 통해 장애 메트릭이 정상화되었는지 확인
Alertmanager의 resolved 상태와 Slack 알림을 통해 장애 대응 흐름이 종료되었는지 검증

주요 Alert Rule 구성

• TalkingPodCrashLooping

- 감지 조건 → talking-dev 네임스페이스의 Pod 컨테이너가 CrashLoopBackOff 상태에 진입
- 선정 이유
 - i. 애플리케이션 실행 오류, 환경변수 누락, Secret 설정 오류, DB/RabbitMQ 연결 실패 등으로 인해 Pod가 반복적으로 재시작되는 상황은 실제 서비스 장애로 이어질 가능성이 높음
 - ii. Kubernetes 운영 중 가장 자주 발생하고 원인 분석이 필요한 장애 유형

• TalkingPodHighRestartRate

- 감지 조건 → talking-dev 네임스페이스의 컨테이너 재시작 횟수가 10분 동안 3회 이상 증가하면 감지
- 선정 이유
 - i. Pod가 완전히 CrashLoopBackOff 상태로 고정되지 않더라도, 간헐적인 예외나 리소스 부족으로 재시작 반복 가능성
 - ii. 사용자가 체감하기 전에는 놓치기 쉬운 장애 전조이므로 조기 탐지를 위해 선정

• TalkingPodNotReady

- 감지 조건 → talking-dev 네임스페이스의 Pod가 Failed, Pending, Unknown 상태로 일정 시간 유지되면 감지
- 선정 이유
 - i. Pod가 Running 상태가 아니거나 스케줄링되지 못하는 상황은 서비스 가용성 저하로 이어질 가능성
 - ii. 리소스 부족, 이미지 Pull 실패, 노드 문제, PVC/Secret 오류 등 다양한 Kubernetes 운영 이슈가 Pod 비정상 상태로 드러나기 때문에 기본적인 서비스 상태 감시 룰로 선정

• TalkingServiceSlowResponse

- 감지 조건 → talking-dev-ingress로 들어오는 ALB Target 평균 응답 시간이 5분 기준 2초를 초과하고, 해당 상태가 2분 이상 지속되면 감지
- 선정 이유
 - i. Pod가 정상 Running 상태여도 실제 사용자가 느끼는 응답 지연은 서비스 장애로 이어질 수 있음
 - ii. CPU/Memory 지표만으로는 확인하기 어려운 애플리케이션 성능 저하를 ALB 응답 시간 기준으로 감지 가능
 - iii. 트래픽 증가, DB 지연, 외부 API 지연, 애플리케이션 내부 병목 등 사용자 체감 장애를 조기에 파악하기 위해 선정

• TalkingHighHttp5xxErrorRate

- 감지 조건 → talking-dev-ingress에서 발생한 Backend 5xx 에러 비율이 5분 기준 5%를 초과하고, 해당 상태가 2분 이상 지속되면 감지
- 선정 이유
 - i. 5xx 에러는 사용자의 요청이 서버 내부 오류로 실패했다는 의미이므로 서비스 장애 판단에 직접적인 지표가 됨
 - ii. 단순 에러 개수가 아니라 전체 요청 대비 비율로 판단하여 일시적인 소량 오류보다 실제 장애 가능성이 높은 상황을 감지 가능
 - iii. 애플리케이션 예외, DB 연결 실패, RabbitMQ 연동 실패, 배포 오류 등 주요 백엔드 장애를 빠르게 탐지하기 위해 선정

• TalkingPodMemorySquatting

- 감지 조건 →talking-dev 네임스페이스의 Pod 메모리 사용량이 설정된 Memory Limit의 85%를 초과하고, 해당 상태가 5분 이상 지속되면 감지
- 선정 이유
 - i. 메모리 사용량이 Limit에 가까워지면 OOMKilled로 Pod가 종료될 가능성이 높아 사전 감지가 필요함
 - ii. Pod가 아직 재시작되지 않았더라도 메모리 누수나 비정상적인 메모리 증가를 조기에 확인 가능
 - iii. 장애 발생 후 대응이 아니라 장애로 이어질 가능성이 높은 리소스 압박 상태를 선제적으로 탐지하기 위해 선정

문제 해결 및 개선 사항

ai-error-analyzer 가용성 확보 방안 검토

- **문제 상황**
ai-error-analyzer는 다른 서비스 장애를 감지하고 분석하는 역할을 수행하기 때문에, chat-service나 notification-service에 장애가 발생하더라도 독립적으로 살아 있어야 함. 하지만 모든 Pod가 같은 EKS 워커 노드의 CPU/Memory 자원을 공유하기 때문에, chat-service에 과부하가 발생하면 같은 노드에 위치한 ai-error-analyzer도 리소스 압박을 받을 수 있음
- **검토한 해결 방안**
 - a. 리소스 Request/Limit 명시
 - i. ai-error-analyzer에 CPU/Memory request와 limit을 명시하여 Kubernetes Scheduler가 최소 자원을 보장하도록 구성
 - ii. chat-service에도 limit을 설정하면 특정 서비스가 노드 자원을 과도하게 점유하는 것을 방지
 - b. Replica 및 Pod 분산 배치
 - i. ai-error-analyzer를 2개 이상의 replica로 운영하고, topologySpreadConstraints 또는 PodAntiAffinity를 적용하여 서로 다른 워커 노드에 분산 배치
 - ii. 단일 Pod 또는 단일 노드 장애에도 분석 기능을 유지
 - c. 전용 워커 노드그룹 분리
 - i. ai-error-analyzer 전용 EKS Node Group을 생성하고 taint/toleration, nodeSelector를 적용하여 분석 서비스만 별도 노드에서 실행
 - ii. 애플리케이션 트래픽 과부하와 장애 분석 시스템을 강하게 격리
- **선택한 방향**
 - 현재 프로젝트 규모에서는 전용 워커 노드그룹을 구성하기보다, 우선 ai-error-analyzer의 리소스 request/limit을 명시하고 replica 및 Pod 분산 배치를 통해 가용성을 확보하는 방향을 선택
 - 전용 노드그룹은 안정성 측면에서는 가장 강력하지만, 추가 EC2 비용과 Terraform/EKS 운영 복잡도가 증가하기 때문에 향후 운영 환경에서 필요성이 확인될 경우 확장하는 방식으로 검토
- **개선 효과**
ai-error-analyzer가 다른 서비스의 과부하로부터 영향을 덜 받도록 최소 자원을 보장할 수 있으며, Pod 분산 배치를 통해 단일 노드 장애에도 분석 파이프라인을 유지할 수 있는 기반을 마련. 또한 전용 노드그룹 분리 가능성을 설계 단계에서 검토하여, 향후 운영 규모가 커졌을 때 장애 분석 시스템을 더 강하게 격리할 수 있는 확장 방향을 확보.

Alert 중복 방지 및 쿨다운 처리

- **문제 상황**
 - Pod 장애가 반복적으로 발생할 경우 동일한 Alert가 짧은 시간 안에 여러 번 firing될 가능성 존재
 - 특히 CrashLoopBackOff나 반복 재시작 장애는 같은 Pod에서 지속적으로 발생할 수 있기 때문에, Alertmanager가 ai-error-analyzer로 여러 번 Webhook을 전송하고 Slack 알림도 반복적으로 전송될 가능성이 있음
 - 이 경우 운영자는 동일한 장애 알림을 여러 번 받게 되어 알림 피로도가 증가하고, 실제로 중요한 장애를 구분하기 어려워질 수 있음
- **해결 방향**
 - a. Alertmanager에서 전달되는 Alert의 fingerprint 값을 기준으로 동일 Alert 여부를 식별하도록 설계
 - b. ai-error-analyzer는 최근 처리한 Alert fingerprint를 일정 시간 동안 저장하고, 같은 fingerprint가 쿨다운 시간 내에 다시 들어올 경우 Slack 알림을 생략하거나 기존 Slack thread에 후속 메시지로 묶어 전송하는 방식으로 개선
- **적용 방안**
 - Alert fingerprint 기준으로 중복 Alert 식별
 - 일정 시간 내 동일 fingerprint 재수신 시 Slack 재전송 제한
 - Alert 상태가 resolved 된 뒤 다시 firing 되면 새로운 장애로 처리
 - 장기적으로는 Slack thread를 활용해 같은 장애의 진행 상황을 하나의 대화 흐름으로 관리
- **개선 효과**
 - 동일 장애에 대한 반복 알림을 줄여 Slack 채널의 노이즈 감소
 - 운영자는 같은 장애의 발생, 재발, 복구 흐름을 더 명확하게 파악할 수 있으며, 알림 피로도를 줄이고 실제 대응이 필요한 장애에 집중할 수 있음

추후 개선사항

- **Kubernetes Events** 및 **Previous Logs** 수집

현재는 장애 Pod의 기본 로그를 중심으로 분석 → CrashLoopBackOff나 OOMKilled 같은 장애는 현재 로그보다 이전 컨테이너 로그와 Kubernetes Events에 원인이 더 잘 드러나는 경우가 많기 때문에, 추후 previous logs, Pod Events, describe 정보까지 함께 수집하도록 개선

- **ai-error-analyzer** 가용성 향상

ai-error-analyzer는 장애 상황에서 반드시 살아 있어야 하는 운영 보조 서비스 → 추후에는 replica 수를 2개 이상으로 늘리고, topologySpreadConstraints 또는 PodAntiAffinity를 적용해 여러 워커 노드에 분산 배치. 필요 시 전용 EKS Node Group을 구성하여 애플리케이션 장애와 분석 시스템을 더 강하게 격리

- **AlertRule** 확대

현재는 장애 발생 알림 중심으로 동작 → 추후에 Alertmanager의 resolved 이벤트를 활용하여 장애가 복구되었을 때 Slack에 복구 메시지를 전송하고, 장애 발생부터 복구까지의 시간을 기록

- **장애 분석 결과 저장**

현재 생성된 런북은 Slack 메시지로만 전달 → 추후에 Alert fingerprint, 발생 시각, 복구 시각, Pod 정보, AI 분석 결과를 DB나 S3에 저장하여 장애 회고, 통계 분석, 재발 방지 문서화에 활용

- **CloudWatch / EventBridge 연동**

현재는 Kubernetes 내부 장애를 Prometheus 기반으로 감지 → 추후에는 CloudWatch Alarm, EventBridge, Lambda를 연동하여 RDS, ALB, NAT Gateway, EKS Control Plane 등 AWS 리소스 장애도 ai-error-analyzer로 전달 → Kubernetes 장애뿐 아니라 클라우드 인프라 장애까지 하나의 ErrorOps 파이프라인에서 분석 가능

기능 예시

- 에러 발생 후 Prometheus Alert Manager가 이를 감지 ai-error-analyzer에게 alert 전송
- ai-error-analyzer가 gemini api로 원인 분석 후 프롬프트 작성 → slack 전송
- Slack에 2명이상 승인시 로컬 codex-runner가 이를 감지 메세지 수신 후 codex cli로 실행

**talkking-codex-runner** 오픈 어제 오후 4:21

[CODEX_TASK]

id: app-readme-pr-test-003

title: app README 테스트 변경 PR 생성

severity: info

[CODEX_TASK].txt ▾

1

[CODEX_TASK]

2

id: app-readme-pr-test-003

3

title: app README 테스트 변경 PR 생성

4

severity: info

5

6

 2



2개의 댓글

**talkking-codex-runner** 오픈 어제 오후 4:21

Task `app-readme-pr-test-003` approved by 1/1. Starting local runner.

Codex runner completed.

Codex execution completed.

Prompt: C:\CE\talkKing\team2-talkKing-codex-runner\prompts\app-readme-pr-test-003.md

Log: C:\CE\talkKing\team2-talkKing-codex-runner\logs\app-readme-pr-test-003.log

- codex cli는 에러가 난 부분을 수정하고 PR생성
- 개발자는 이를 확인하고 merge

chore: add codex runner readme test #76 

 Merged

[alpacadash](#) merged 2 commits into `dev` from `chore/codex-runner-readme-test`  yesterday

 Conversation 0

 Commits 2

 Checks 5

 Files changed 2

**alpacadash** commented [yesterday](#)

Member 

No description provided.



 **alpacadash** added 2 commits [yesterday](#)

 chore: add codex runner readme test [24d9b98](#)

 fix: prompt to pr not draft pr [3699d77](#) 

 **KK-Hyeok** approved these changes [yesterday](#) [View reviewed changes](#)

 **alpacadash** merged commit `03849fa` into `dev` [yesterday](#) [View details](#) [Revert](#)

5 checks passed